



MTU

Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

STAT8010: Intro to R for Data Science

Chapter 2.

Useful Functions & Pattern Recognition

Francisco Hernández

francisco.hernandez@cit.ie



CIT
CORK
INSTITUTE OF
TECHNOLOGY
INSTITIÚID TEICNEOLAÍOCHTA CHORCAI

Data Science
and Analytics

www.mtu.ie

Lecture Content

- A collection of **useful functions** in R
- Investigate how **R** can be used for mathematical and statistical **calculations**
- Some basic **data** frame **manipulation**
- Regular **expressions** in R
- Pattern **recognition**
- Pattern recognition in R for **data cleansing** purposes

...some useful commands...

- `Help(topic)`
 - `help(plot)` #opens documentation on plot functions
- `help.search("topic")` #search the help system
- `dir()` #show files in the current directory
- (same as `list.files()`)

...some useful commands...

- `install.packages("name")` # install a package from repository
- `library(name)` # load add-on packages
- `print(a, ...)` # prints its arguments
- `cat(...)` # Outputs the objects, concatenating the representations. [performs much less conversion than `print()`]

...some useful commands...

- `from:to` →
 - `1:4` #outputs 1,2,3,4
 - `seq(from,to)` # generates a sequence
- `paste(...)` # Concatenate vectors after converting to “char”.
 - `paste("Hi", "Hi")` # outputs "Hi Hi"
 - `"sep="` string to separate terms (a single space is the default)
 - `paste("Hi", "Hi", sep=' ')` # outputs "Hi Hi"

`substr(x,start,stop)` substrings in a character vector;

- `substr("Hello",2,4)` # outputs "ell"

Mathematical & Statistical functions

$\sin(x)$ # sine of x

$\cos(x)$ # cosine of x

$\tan(x)$ # tangent of x

$\arcsin(x)$ # arcsin of x

$\arccos(x)$ # arccos of x

$\arctan(x)$ # arctan of x

$\log(x)$ # natural log of x , $\ln(x)$

$\log_{10}(x)$ # log base 10, $\log_{10}(x)$

$\exp(x)$ # e^x

$\max(x)$ # maximum in x

$\min(x)$ # minimum in x

Mathematical & Statistical functions

`mean(x)` # mean of x

`median(x)` # median of x

`sd(x)` # standard deviation of x

`var(x)` # variance of x

`IQR(x)` # Interquartile range of x

`cov(x)` # covariance of x

Mathematical & Statistical functions

`range (x)` # range of x

`sum (x)` # sum of x

`diff (x)` # First Differences in x

`diff(x, lag = 1, differences = 1, ...)`

`quantile (x)` # 0, 25, 50, 75 & 100% quantiles of x

`quantile (x, p)` # p% quantiles of the values x

Plotting functions

`plot(x)` # plot of x (on the y -axis) ordered on the x -axis

`plot(x, y)` # bivariate plot of x (on x -axis) and y (on y -axis)

`hist(x)` # histogram of the frequencies of x

`barplot(x)` # histogram of the values of x ; use `horiz=TRUE` for horizontal bars

`matplot(x, y)` # plot of columns of matrices

`boxplot(x)` # "box-and-whiskers" plot

`stem(x)` # stem&leaf plot

`dotchart(x)` # if x is a data frame, stacked plots line-by-line and column-by-column.

Plotting functions

`dotchart(x)` # if x is a data frame, stacked plots line-by-line and column-by-column.

`pie(x)` # circular pie-chart

`sunflowerplot(x, y)` # id. than `plot()` but the points with similar coordinates are drawn as flowers which petal number represents the number of points

`stripchart(x)` # plot of the values of x on a line (an alternative to `boxplot()` for small sample sizes)

Plotting functions

`points(x, y)` # adds points to a plot (the option `type=` can be used)

`lines(x, y)` # as with points but with lines

(use of `matpoints` & `matlines` for `matplot` of matrices)

`text(x, y, labels)` # adds text given by labels at coordinates (x,y)

`abline(a,b)` # draws a line of slope b and intercept a.

`abline(h=y)` draws a horizontal line at ordinate y

`abline(v=x)` # draws a vertical line at abscissa x.

`abline(lm.obj)` draws the regression line given by the model "lm.obj"

Plotting functions

`rect(x1, y1, x2, y2)` # draws a rectangle which left, bottom, right and top limits are x1, y1, x2, and y2,
`legend(x, y, legend)` # adds the legend at the point (x,y) with the symbols given by legend
`title()` adds a title and optionally a sub-title
`axis(side, vect)` # adds an axis at the bottom (side=1), on the left (2), at the top (3), or on the right (4);

Data manipulation

`attach(dataframe)` # attaches data frame to R workspace -
allows user to easily call variables from data set

`detach(dataframe)` # detaches data frame (or library) from R
workspace

`subset(dataframe, ...)` # returns a smaller data frame
subset by options ...

e.g. `subset(iris, Species == "virginica")`

e.g. `subset(iris, Petal.Length > 6)`

Data manipulation

`order(vec, ...)` # returns a vector containing the sorted order of the vector `vec`

e.g. to **sort** the iris data frame by ascending values of `Petal.Length`:

```
iris[order(iris$Petal.Length),]
```

To **sort by descending** values of `Sepal.Length`, use the **- sign**:

```
iris[order(-iris$Sepal.Length),]
```

`sort(vec, ...)` # returns a vector containing the sorted order of the values in the vector `vec`

Regular expressions

Regular expressions are a concise and flexible tool for **describing patterns** in strings.

Detailed **documentation** on regular expressions is available in R by running:

```
?regex
```

The main **advantage** of regular expressions is to alter character strings.

There are many different **functions** for doing this, see:

```
?grep
```

This can have useful applications when **cleaning** a data set or when you need to alter scrapped text.

Using Regular expressions

`grep(...)` searches for a regular **expression** in a text vector and returns a vector with the **positions** of the elements where the expression is matched.

`grep1(...)` [`grep Logical`] is similar to `grep`, it returns a **logical** vector whose element is true when **expression** is matched.

`gregexpr(...)` similar to `grep` more complex output in a **list**

`sub(...)` performs **replacement** on the **first match** only.

`gsub(...)` performs replacement on **all matches** (or some matches as specified in the options).

`glob2rx(...)` [regular expression generator] **changes** wildcard (aka globbing) **patterns** into the corresponding regular **expressions**.

Regular expression Syntax



MTU

Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

Syntax	Description
\d	Digit, 0,1,2 ... 9
\D	Not Digit
\s	Space
\S	Not Space
\w	Word
\W	Not Word
\t	Tab
\n	New line
^	Beginning of the string
\$	End of the string
\	Escape special characters, e.g. \ is "\", \+ is "+"
	Alternation match. e.g. /(e d)n/ matches "en" and "dn"
.	Any character, except \n or line terminator
[ab]	a or b
[^ab]	Any character except a and b
[0-9]	All Digit
[A-Z]	All uppercase A to Z letters
[a-z]	All lowercase a to z letters
[A-z]	All Uppercase and lowercase a to z letters

Syntax	Description
i+	i at least one time
i*	i zero or more times
i?	i zero or 1 time
i{n}	i occurs n times in sequence
i{n1,n2}	i occurs n1 - n2 times in sequence
i{n1,n2}?	non greedy match, see above example
i{n,}	i occurs >= n times
[:alnum:]	Alphanumeric characters: [:alpha:] and [:digit:]
[:alpha:]	Alphabetic characters: [:lower:] and [:upper:]
[:blank:]	Blank characters: e.g. space, tab
[:cntrl:]	Control characters
[:digit:]	Digits: 0 1 2 3 4 5 6 7 8 9
[:graph:]	Graphical characters: [:alnum:] and [:punct:]
[:lower:]	Lower-case letters in the current locale
[:print:]	Printable characters: [:alnum:], [:punct:] and space
[:punct:]	Punctuation character: !"#\$%&'()*+,-./:;<=>?@[\\]^_`{ }~
[:space:]	Space characters: tab, newline, vertical tab, form feed, carriage return, space
[:upper:]	Upper-case letters in the current locale
[:xdigit:]	Hexadecimal digits: 0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f

Some Basic examples

Swapping entire words

```
x <- c("R Tutorial", "PHP Tutorial", "HTML Tutorial")  
gsub("Tutorial", "Examples", x)
```

capitalising Words

```
gsub("(\\w)(\\w*)", "\\U\\1\\L\\2", "a test of capitalising", perl=TRUE)  
gsub("\\b(\\w)", "\\U\\1", "a test of capitalising", perl=TRUE)
```

More examples

```
mysentence <- "England is Beautiful. England is not the part of EU"  
sub("England", "UK", mysentence)  
#this replaces only the first instance of "England" with "UK"  
  
mysentence <- "England is Beautiful. England is not the part of EU"  
gsub("England", "UK", mysentence)  
#this replaces all instances of "England" with "UK"  
  
mysentence <- "UK is Beautiful. UK is not the part of EU since 2016"  
gsub("[0-9]*", "", mysentence)  
#this will replace all instances of digits/numbers with nothing.
```

Pattern Recognition

Pattern recognition involves using statistical/machine learning **models** to detect pattern or irregularities in data.

Can take many forms with different models with different mechanisms.

Can be implemented easily in **software** such as R or Python

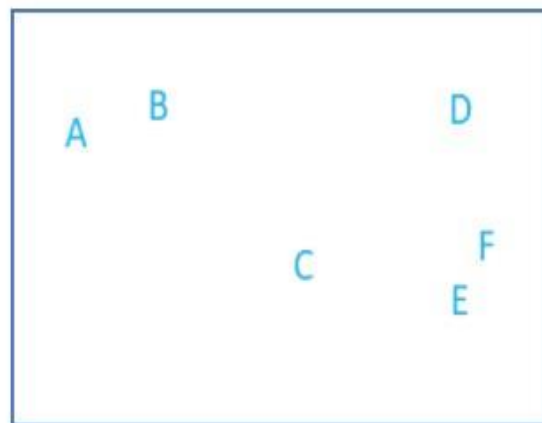
Pattern Recognition & Data Cleaning



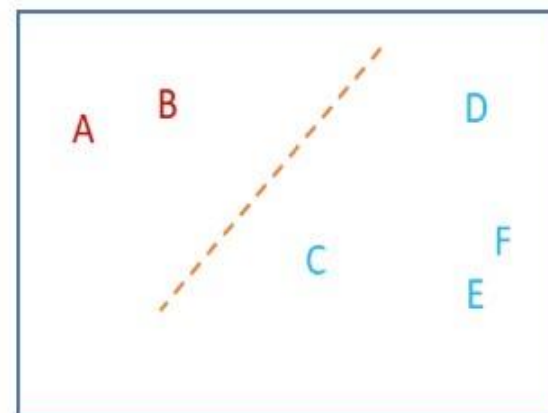
Data **cleansing** consists of various tasks such as changing the format of data but also detecting erroneous values.

One way to do this involved sourcing a data set and applying pattern recognition **algorithms** to detect values that seem likely to be incorrect.

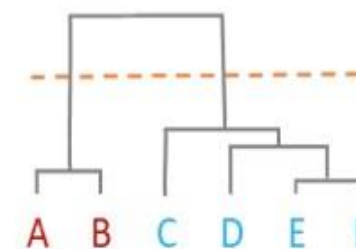
Understanding a Basic Dendrogram



Dendrogram



Dendrogram



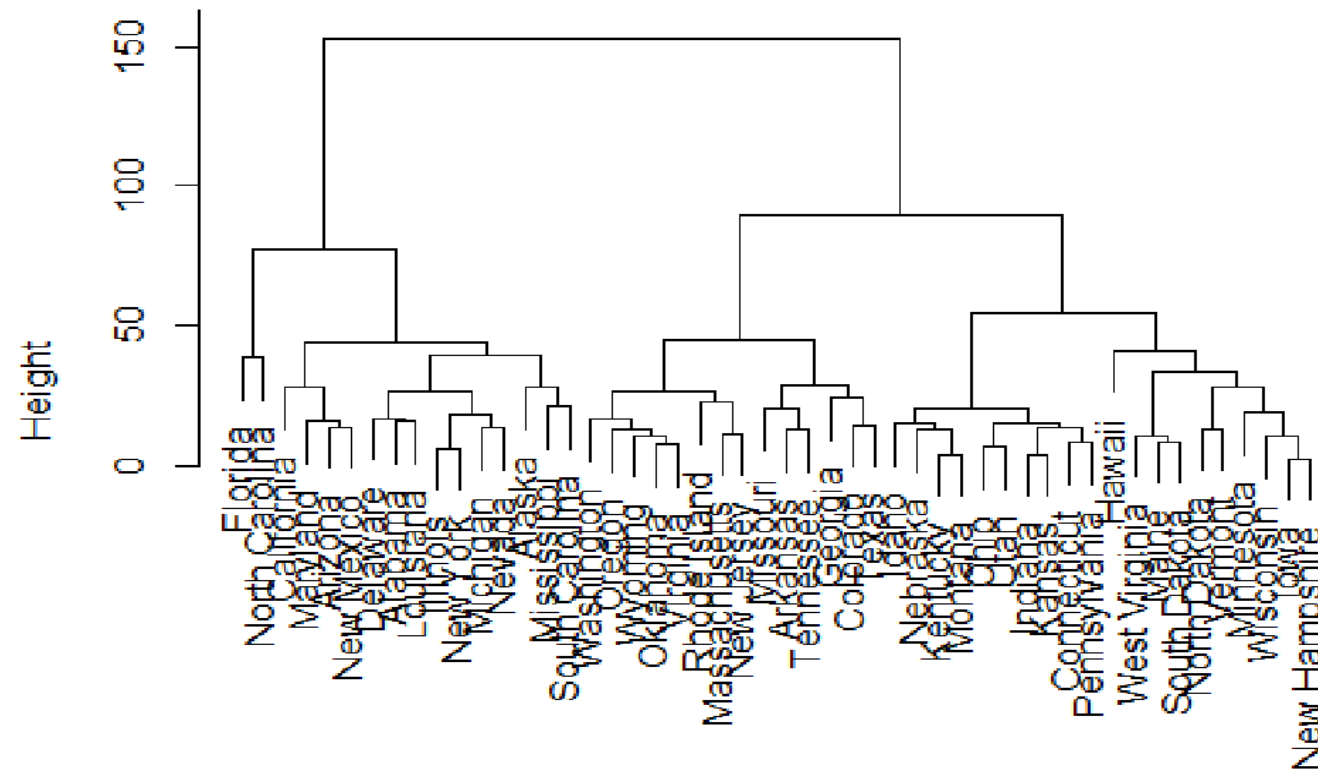
Example: Cluster Analysis



```
hc <- hclust(dist(USArrests), "ave")
```

```
plot(hc)
```

Cluster Dendrogram

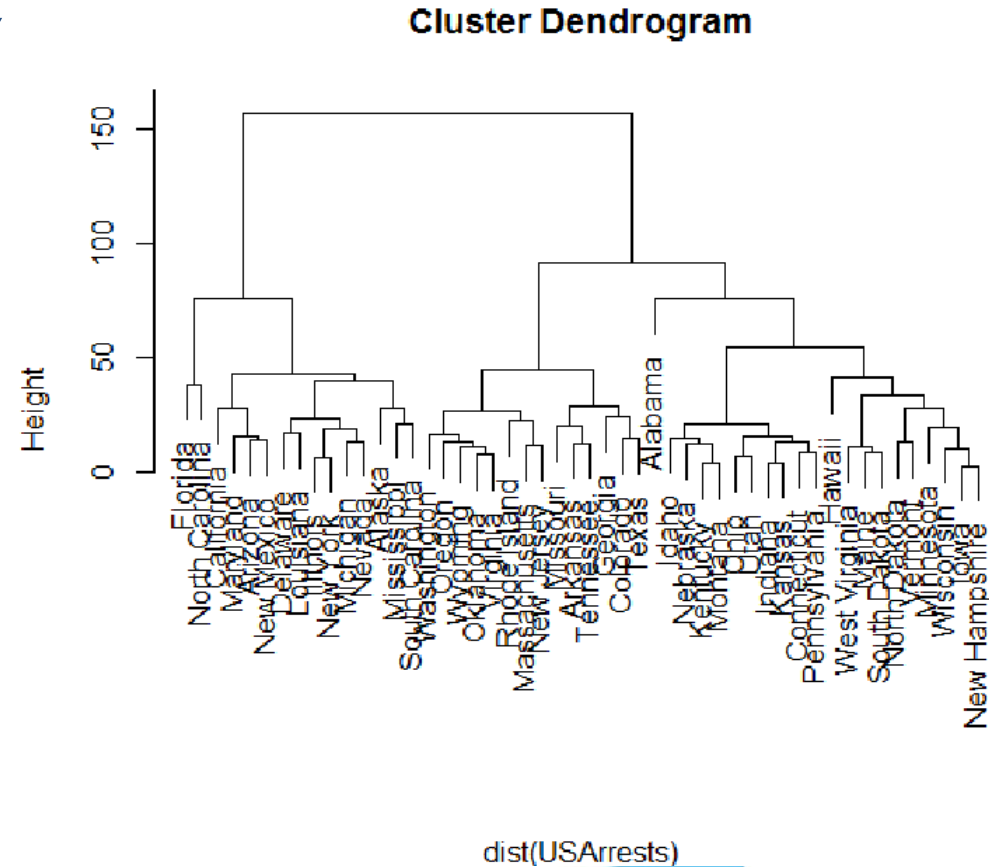


Cluster Analysis & Data Cleaning

```
USArrests[1,]=50
```

```
hc <- hclust(dist(USArrests),
```

```
plot(hc)
```



PCA as Cluster tool

```
xx = iris[,1:4]
```

```
xx
```

```
prcomp(xx) # inappropriate
```

```
prcomp(xx, scale. = TRUE)
```

```
par(mfrow=c(1,2))
```

```
plot(prcomp(xx))
```

```
plot(prcomp(xx, scale. = TRUE))
```

```
summary(prcomp(xx))
```

```
summary(prcomp(xx, scale. = TRUE))
```

```
biplot(prcomp(xx), main="RAW DATA")
```

```
biplot(prcomp(xx, scale. = TRUE), main="SCALED DATA")
```

